

자연어처리와 컴퓨터비전 심층학습 5~6장(~TF-IDF)

📅 날짜	2026년 5월 12일
🕒 발표팀	4팀
☀️ 상태	시작 전
🏷️ 태그	10주차

5장. 토큰화 (Tokenization)

▶ 자연어 처리(Natural Language Processing, NLP) 개요

자연어(Natural Language)란 인공적으로 만들어진 프로그래밍 언어와 달리, 사람들이 일상적인 언어 활동을 위해 자연히 만들어진 언어를 의미한다.

자연어 처리(NLP)는 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술로, 인공지능의 하위 분야 중 하나이다. 모델 개발 시 해결해야 할 주요 문제:

- 모호성(Ambiguity): 단어나 구가 맥락에 따라 여러 의미를 가짐
- 가변성(Variability): 사투리, 강세, 신조어, 작문 스타일 등으로 인한 언어 다양성
- 구조(Structure): 문장의 구문과 문법적 의미를 이해해 의미를 추론해야 함

핵심 용어:

- 말뭉치(Corpus): 자연어 처리 모델 훈련·평가에 사용되는 대규모 자연어 데이터
- 토큰(Token): 말뭉치보다 작은 단위로 개별 단어, 구두점, 기타 의미 단위
- 토큰화(Tokenization): 텍스트를 토큰으로 나누는 과정
- 토큰나이저(Tokenizer): 텍스트를 토큰으로 나누는 알고리즘 또는 소프트웨어

입력: "형태소 분석기를 활용하면 ..."

출력: ['형태소', '분석기', '를', '활용하면', '...']

▶ 단어 토큰화 (Word Tokenization)

텍스트를 단어(Word) 단위로 나누는 토큰화 방법.

띄어쓰기, 문장 부호, 대소문자 등의 구분자를 활용한다.

품사 태깅, 개체명 인식, 기계 번역 등에서 널리 사용되는 가장 일반적인 방법.

[장점]

- 구현이 쉽고 직관적

[단점]

- '최고!'와 '최고'를 다른 토큰으로 처리하는 문제
- 형태가 다른 단어(그는/그에게/그녀는)를 별도 토큰으로 처리 → 어휘 사전 비대화
- 미등록 단어(OOV, Out-Of-Vocabulary) 처리 어려움

구현: split() 메서드 사용 (공백 기준 분리)

▶ 글자 토큰화 (Character Tokenization)

텍스트를 글자(Character) 단위로 나누는 토큰화 방법.

언어 모델링과 같은 시퀀스 예측 작업에 활용된다.

[장점]

- 상대적으로 적은 어휘 사전 크기
- OOV 문제를 크게 줄일 수 있음

[단점]

- 입력 시퀀스 길이가 길어짐 (단어 토큰화 13 → 글자 토큰화 53)
- 개별 토큰이 의미를 갖지 않아 모델이 의미를 조합해야 함
- 한글은 자음+모음 조합이므로, 자소(자모) 단위 토큰화가 추가로 필요

자소 단위 토큰화(jamo 라이브러리):

- 완성형 한글 → 조합형(초성·중성·종성) 변환 후 자소 단위 분리
- h2j(): 완성형 → 조합형 변환
- j2hcj(): 조합형 → 자소 단위 분리

▶ 형태소 토큰화 (Morpheme Tokenization)

텍스트를 형태소(Morpheme) 단위로 나누는 방법.

언어의 문법과 구조를 고려해 단어를 분리하고 의미 있는 단위로 분류한다.

한국어는 교착어(Agglutinative Language)로, 어근에 다양한 접사와 조사가 조합되어 하나의 낱말을 이룬다. 예: "그는", "그에게", "그녀는" → 형태소: 그, -는/-에게/-녀

형태소 종류:

- 자립 형태소: 독립적으로 사용 가능 (예: 그, 나, 인사)
- 의존 형태소: 다른 형태소와 결합해야 사용 (예: -는, -이기, -를, 했-, -다)

형태소 어휘 사전 (Morpheme Vocabulary):

형태소 정보를 포함하는 사전으로, 각 단어가 어떤 형태소들의 조합으로 이루어졌는지에 대한 정보를 담고 있다. 형태소 분석 작업에서 매우 중요한 역할을 한다.

[형태소 분석의 한계]

- 신조어: '돈쫄내다' → 형태소 분석기가 적절히 분리하지 못함
- 외래어: '시보리' → '시'+ '보리'로 잘못 분리
- 전문용어, 고유어가 많은 데이터에서 성능 저하

1. KoNLPy

한국어 자연어 처리를 위한 라이브러리. 명사 추출, 형태소 분석, 품사 태깅 등 제공.
Java 기반이므로 JDK 설치 필요.

지원하는 형태소 분석기:

- Okt (Open Korean Text): SNS 텍스트 기반으로 개발
- Kkma (꼬꼬마): 국립국어원 세종 말뭉치 기반
- Komoran (코모란)
- Hannanum (한나눔)
- Mecab (메캅, 윈도우 미지원)

Okt 주요 메서드:

okt.nouns(sentence) → 명사 추출, 리스트 반환

okt.phrases(sentence) → 어절·구 단위 추출

okt.morphs(sentence) → 형태소 추출

okt.pos(sentence) → 품사 태깅, (형태소, 품사) 튜플 리스트 반환

Kkma 주요 메서드:

kkma.nouns(sentence) → 명사 추출

kkma.sentences(sentence) → 문장 분리

kkma.morphs(sentence) → 형태소 추출

kkma.pos(sentence) → 품사 태깅

1. NLTK (Natural Language Toolkit)

자연어 처리를 위한 영어 중심 라이브러리. 네덜란드어, 프랑스어, 독일어 등도 지원.

기능: 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등

사용 전 모델 다운로드 필요:

nlTK.download("punkt") → 통계 기반 토큰라이저
nlTK.download("averaged_perceptron_tagger") → 퍼셉트론 기반 품사 태거

주요 함수:

tokenize.word_tokenize(sentence) → 단어 토큰화 (공백·구두점 기준)
tokenize.sent_tokenize(sentence) → 문장 토큰화 (마침표·느낌표·물음표 기준)
tag.pos_tag(word_tokens) → 품사 태깅 (35개 품사 태그 지원)

Averaged Perceptron Tagger 주요 품사 태그:

CC: 접속사 / DT: 관형사 / JJ: 형용사 / NN: 단수 명사 / NNS: 복수 명사
VB: 기본형 동사 / VBD: 과거형 동사 / RB: 부사 / PRP: 인칭 대명사

1. spaCy

사이언(Cython) 기반 오픈소스 NLP 라이브러리.

NLTK와의 차이: 빠른 속도와 높은 정확도를 목표로 하는 머신러닝 기반.

24개 이상의 언어로 사전 학습된 모델 제공.

사용법:

nlp = spacy.load("en_core_web_sm") → 모델 로드
doc = nlp(sentence) → 처리 결과 doc 객체
for token in doc: → token 객체 순회

token 객체 주요 속성:

token.pos_ → 기본 품사
token.tag_ → 세분화 품사
token.text → 원본 텍스트
token.vector → 벡터값

▶ 하위 단어 토큰화 (Subword Tokenization)

단어 토큰화와 글자 토큰화의 중간 방식. 자주 등장하는 단어는 하나의 토큰으로, 드물게 등장하는 단어는 여러 하위 단어(Subword)의 조합으로 표현한다.

장점: OOV 문제를 효과적으로 해결하고 적절한 어휘 사전 크기 유지

종류: 바이트 페어 인코딩(BPE), 워드피스(WordPiece), 유니그램(Unigram)

1. 바이트 페어 인코딩 (Byte Pair Encoding, BPE)

다이그램 코딩(Digram coding)이라고도 함.

텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍을 찾아 부호화하는 압축 알고리즘.

알고리즘 동작 방식:

- 1) 모든 단어를 글자 단위로 분리하여 빈도 사전 구성
- 2) 가장 자주 등장하는 글자 쌍을 찾아 하나의 토큰으로 병합
- 3) 어휘 사전에 새 토큰 추가
- 4) 더 이상 병합할 쌍이 없거나 어휘 사전 크기에 도달할 때까지 반복

예시:

빈도 사전: {'low':5, 'lower':2, 'newest':6, 'widest':3}

글자 분리: l-o-w(5), l-o-w-e-r(2), n-e-w-e-s-t(6), w-i-d-e-s-t(3)

→ 가장 빈번한 쌍('e','s') 병합 → 'es' 토큰 생성

→ 반복 후 어휘 사전: ['l','o','w','e','r','n','s','t','es','est','lo','low','ne','new',
'newest','wi','wid','widest'] 등

결과: 'newer', 'wider' 같은 미등록 단어도 기존 하위 단어로 분리 가능 (OOV 해결)

실습 라이브러리: SentencePiece (Google 개발)

- 바이트 페어 인코딩, 워드피스, 유니그램 알고리즘 지원
- 유니코드 기반, 다양한 하이퍼파라미터 설정 가능
- 공백을 특수문자 '▯'(U+2581)로 표현

SentencePieceTrainer 주요 매개변수:

input : 말뭉치 텍스트 파일 경로
model_prefix : 모델 파일 이름
vocab_size : 어휘 사전 크기

character_coverage: 말뭉치 내 글자 중 다룰 수 있는 글자 비율
model_type : 토큰나이저 알고리즘 (bpe / unigram / char / word)
max_sentence_length: 최대 문장 길이
unk_id : OOV 매핑 unk 토큰 ID (기본값 0)
bos_id : 문장 시작 bos 토큰 ID (기본값 1)
eos_id : 문장 끝 eos 토큰 ID (기본값 2)

특수 토큰: <unk>(미등록 단어), <s>(문장 시작), </s>(문장 끝)

1. 워드피스 (WordPiece)

바이트 페어 인코딩과 유사하지만, 빈도 기반이 아닌 확률 기반으로 글자 쌍을 병합.

글자 쌍 병합 점수 공식:

$$\text{score}(x, y) = \text{freq}(xy) / (\text{freq}(x) \times \text{freq}(y))$$

→ xy 조합 빈도를 x·y 각각의 빈도 곱으로 나눈 값

→ 단독으로도 자주 등장하는 글자보다, 함께 나올 때 의미 있는 쌍을 우선 병합

[BPE와의 차이]

- BPE: 글자 쌍 빈도만 고려
- WordPiece: 글자 쌍의 조건부 확률(점수)을 계산해 가장 높은 쌍을 선택

실습 라이브러리: Tokenizers (Hugging Face)

- Normalization(정규화): 불필요한 공백 제거, 대소문자 변환, 유니코드 정규화 등

- Pre-tokenization(사전 토큰화): 공백·구두점 기준으로 먼저 분리

- 학습 결과는 JSON 파일로 저장 (정규화·사전 토큰화 메타데이터 포함)

주요 클래스:

Tokenizer(WordPiece()) → 워드피스 모델 토큰나이저 생성

tokenizer.normalizer → 정규화 방식 설정

tokenizer.pre_tokenizer → 사전 토큰화 방식 설정

tokenizer.train(["corpus.txt"]) → 모델 학습

tokenizer.encode(sentence).tokens → 토큰화 결과

tokenizer.encode(sentence).ids → 정수 인코딩 결과

tokenizer.decode(ids) → 정수 → 문장 변환

Pre-tokenizer 종류:

Whitespace() : 단어 경계·공백 기준 분리

ByteLevel() : GPT-2 방식, 문자열을 그래픽 기호에 매핑

Punctuation() : 구두점 기준 분리

Metaspace() : 공백 기준 분리, 공백을 '_'로 치환

Split(pattern, behavior): 정규표현식 기반 분리

6장. 임베딩 (Embedding) — TF-IDF까지

▶ 텍스트 벡터화 개요

컴퓨터는 텍스트 자체를 이해할 수 없으므로, 텍스트를 숫자로 변환하는 텍스트 벡터화(Text Vectorization) 과정이 필요하다.

1. 원-핫 인코딩 (One-Hot Encoding)

문서에 등장하는 각 단어를 고유한 색인 값으로 매핑한 후, 해당 색인 위치를 1로, 나머지를 모두 0으로 표시하는 방식.

예: 말뭉치 ["I", "like", "apples", "bananas"]

색인: 0 1 2 3

"I like apples" → [1, 1, 1, 0]

"I like bananas" → [1, 1, 0, 1]

2. 빈도 벡터화 (Frequency Vectorization)

문서에서 단어가 등장한 빈도수를 벡터로 표현하는 방식.

'apples'가 4번 등장하면 해당 값은 4.
원-핫 인코딩은 등장 여부(0/1)만 표현, 빈도 벡터화는 등장 횟수로 표현.

3. Bag of Words (BoW)

문서를 단어의 집합으로 표현하며, 단어 중복을 허용해 빈도를 기록하는 방법.

[공통 단점 — 원-핫, BoW, 빈도 벡터화]

- 벡터의 희소성(Sparsity): 어휘 사전 크기만큼의 차원이 필요하지만 실제 문장에는 극히 일부 단어만 등장 → 대부분 0으로 채워짐 (차원의 저주)
- 단어의 순서/문맥 정보 미반영
- 의미적으로 유사한 문장도 벡터가 다르게 표현될 수 있음

→ 이 문제를 해결하기 위해 워드 임베딩(Word Embedding) 기법 사용
→ Word2Vec, fastText 등: 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현하여 단어 간 관계를 추론

언어 모델 (Language Model)

입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델.
자동 번역, 음성 인식, 텍스트 요약 등 다양한 NLP 분야에 활용.

1. 자기회귀 언어 모델 (Autoregressive Language Model)

입력된 문장들의 조건부 확률을 이용해 다음에 올 단어를 예측.
이전에 등장한 모든 토큰의 정보를 고려해 문장의 문맥을 파악.

수식 (연쇄법칙 적용):

$$P(w_1, w_2, \dots, w_n) = P(w_1) \times P(w_2|w_1) \times P(w_3|w_1, w_2) \times \dots \times P(w_n|w_1, \dots, w_{n-1})$$

특징:

- 출력값이 다시 입력값으로 사용 → '자기회귀(Autoregressive)'라는 이름
- 토큰 분류 문제로 정의됨
- 대표 모델: GPT (문장 앞부분을 입력으로 다음 단어 예측)

GPT 예시:

Input: "GPT는 OpenAI에서 개발한"

Prediction: "인과적 언어 모델입니다."

BERT 예시 (마스크 언어 모델):

Input: "BERT는 [MASK]에서 개발한 [MASK] 언어 모델입니다."

Prediction: Google, 마스크된

1. 통계적 언어 모델 (Statistical Language Model)

언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성·분석.

일반적으로 마르코프 체인(Markov Chain)을 이용해 구현.

마르코프 체인: 이전 상태와 현재 상태 간의 전이 확률로 다음 상태 예측.

빈도 기반 조건부 확률 수식:

$$P(\text{만나서} \mid \text{안녕하세요}) = \frac{\text{빈도(안녕하세요 만나서)}}{\text{빈도(안녕하세요)}} \\ = 700 / 1000 = 0.7$$

[단점]

- 단어 순서와 빈도에만 기초 → 문맥 파악 부족
- 데이터 희소성(Data Sparsity): 한 번도 등장하지 않은 단어나 문장에 대한 확률 예측이 어려움

N-gram

가장 기초적인 통계적 언어 모델. 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급하여 특정 단어 시퀀스가 등장할 확률을 추정.

N값에 따른 명칭:

N=1 → 유니그램 (Unigram)

N=2 → 바이그램 (Bigram)

N=3 → 트라이그램 (Trigram)

N≥4 → N-gram

예시 문장: "안녕하세요 만나서 진심으로 반가워요"

Unigram : [안녕하세요], [만나서], [진심으로], [반가워요]

Bigram : [안녕하세요 만나서], [만나서 진심으로], [진심으로 반가워요]

Trigram : [안녕하세요 만나서 진심으로], [만나서 진심으로 반가워요]

N-gram 조건부 확률 수식:

$P(w_t | w_{t-n+1}, \dots, w_{t-1})$

→ N-gram 언어 모델은 N-1개의 이전 토큰만 고려해 확률 계산

N-gram의 활용:

- 소규모 데이터에서 연속된 문자열 패턴 분석
- 관용적 표현 분석 (예: "입이 무겁다" = "비밀을 잘 지킨다")
- 단어 순서가 중요한 NLP 작업 및 문자열 패턴 분석

TF-IDF (Term Frequency - Inverse Document Frequency)

텍스트 문서에서 특정 단어의 중요도를 계산하는 방법.

BoW(Bag of Words)에 가중치를 부여하는 방법으로, 단어의 상대적 중요도를 통계적으로 평가한다.

1. 단어 빈도 (Term Frequency, TF)

문서 내에서 특정 단어가 등장한 빈도수.

수식:

$TF(t, d) = \text{count}(t, d)$

(t: 단어, d: 문서, count: 단어 t가 문서 d에 등장한 횟수)

특징:

- TF 값이 높을수록 해당 단어가 특정 문서에서 자주 사용됨
- 단, 문서 길이가 길어질수록 TF 값도 증가하는 한계가 있음

1. 문서 빈도 (Document Frequency, DF)

한 단어가 얼마나 많은 문서에 나타나는지를 나타내는 값.

수식:

$DF(t, D) = \text{count}(t \in d : d \in D)$

(D: 전체 문서 집합)

해석:

- DF 높음 → 많은 문서에 등장 → 일반적인 단어, 중요도 낮음
- DF 낮음 → 특정 문서에만 등장 → 특정 문맥에서 사용, 중요도 높음

1. 역문서 빈도 (Inverse Document Frequency, IDF)

전체 문서 수를 문서 빈도로 나눈 후 로그를 취한 값.

단어의 중요도를 보정하는 역할.

수식:

$IDF(t, D) = \log(\text{count}(D) / (DF(t, D) + 1))$

설계 의도:

- 분모에 +1: 특정 단어가 한 번도 등장하지 않을 때 분모가 0이 되는 것을 방지
- 로그 사용: 전체 문서 수를 DF로 나누면 값이 너무 커질 수 있어 로그로 스케일 조정 (예: 10,000개 문서에서 1번 등장한 단어 → $IDF = \log(10,000) \approx 4, 5,000$ 아님)

1. TF-IDF 계산

수식:

$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, D)$

해석:

- 문서 내에서 자주 등장하지만(TF 높음) 전체 문서에서는 드물게 등장하는(IDF 높음) 단어의 TF-IDF 값이 높다 → 해당 문서에서 중요한 단어
- 전체 문서에서 자주 등장하는 단어(the, is, a 등 관용어)는 IDF 값이 낮아 가중치↓

구현 라이브러리: scikit-learn의 TfidfVectorizer

TfidfVectorizer 주요 매개변수:

input : 입력 데이터 형태 ('content'=문자열, 'file'=파일 객체, 'filename'=파일 경로)
 encoding : 텍스트 인코딩 (기본값 'utf-8')
 lowercase : 소문자 변환 여부 (True/False)
 stop_words : 불용어 처리 (분석에 불필요한 단어 제거)
 ngram_range : N-gram 범위 (최솟값, 최댓값)
 (1,1)=유니그램, (1,2)=유니그램+바이그램
 max_df : 최대 문서 빈도 (해당 비율 초과 단어를 불용어 처리)
 min_df : 최소 문서 빈도 (해당 비율 미만 단어를 불용어 처리)
 vocabulary : 미리 구축한 단어 사전 (없으면 자동 구축)
 smooth_idf : IDF 분모에 1을 더해 0 나눗셈 방지 (True 권장)

사용 방법:

```
tfidf = TfidfVectorizer()
tfidf.fit(corpus)           → 학습 (단어 사전 구축)
matrix = tfidf.transform(corpus) → 벡터화
# 또는 한번에: tfidf.fit_transform(corpus)
```

```
matrix.toarray()    → (문서 수) × (단어 수) 넘파이 배열
tfidf.vocabulary_   → 단어 사전 (단어: 색인 딕셔너리)
```

[TF-IDF의 한계]

- 빈도 기반이므로 문장의 순서나 문맥을 고려하지 않음
- 문장 생성같이 순서가 중요한 작업에는 부적합
- 벡터가 단어의 의미(Semantics)를 담고 있지 않음
→ "나는 바나나를 먹었다"와 "그녀가 과일을 섭취한다"의 유사도 계산 불가
- 이러한 한계를 극복하기 위해 Word2Vec 등 밀집 표현(Dense Representation) 사용